Personality-Driven Chatbot - Complete Setup Guide

What This Bot Does

This chatbot has a configurable personality and can respond to chat messages, handle commands, and remember users. It's designed to work on StumbleChat and can be customized to behave however you want it to behave.

Think of it like creating a digital person with their own personality, preferences, and way of talking. The bot can be friendly, sarcastic, helpful, or any combination of traits you design.

Before You Start - What You Need

You need these things installed on your computer before anything else will work:

1. **Node.js** (version 16 or newer) - This is like the engine that runs JavaScript code
2. **A text editor** - Notepad++ (Windows), TextEdit (Mac), or VS Code (any system) work fine
3. **Your bot's login credentials** - Username and password for StumbleChat
4. **The room name** where you want your bot to appear

If you don't have Node.js, download it from nodejs.org and install it. When installation finishes, restart your computer.

Step 1: Set Up Your Project Folder

Create a new folder on your computer called `personality-chatbot` (or whatever name you prefer). This will hold all your bot's files.

Inside that folder, create this exact folder structure:

```
personality-chatbot/
├── src/
│   ├── core/
│   └── modules/
├── config/
└── logs/
```

To create folders: Right-click in your main folder, select "New Folder" (Windows) or "New Folder" (Mac), and type the folder name.

Step 2: Place Your Files

Core Bot File

Put the main PersonalityEngine.js file in the `src/core/` folder. This is your bot's brain.

Module Files (Optional Enhancement Modules)

Put these five files in the `src/modules/` folder:

- `EnhancedCommandHandler.js`
- `MusicSystem.js`
- `MessageProcessor.js`
- `OwnerPrivilegeSystem.js`
- `AdvancedSelfAwareness.js`

These modules add extra features to your bot. The bot works fine without them, but they make it much more capable.

Your folder should now look like this:

```
personality-chatbot/
├── src/
│   ├── core/
│   │   └── PersonalityEngine.js
│   └── modules/
│       ├── EnhancedCommandHandler.js
│       ├── MusicSystem.js
│       ├── MessageProcessor.js
│       ├── OwnerPrivilegeSystem.js
│       └── AdvancedSelfAwareness.js
├── config/
└── logs/
```

Step 3: Create Your Configuration Files

Package.json File

Create a file called `package.json` in your main folder (personality-chatbot). Copy and paste this exactly:

```json
```

```
{
  "name": "personality-chatbot",
  "version": "1.0.0",
  "description": "A personality-driven chatbot for StumbleChat",
  "main": "src/core/PersonalityEngine.js",
  "scripts": {
    "start": "node src/core/PersonalityEngine.js",
    "test": "echo \"No tests specified\" && exit 0"
  },
  "dependencies": {
    "puppeteer": "^21.0.0",
    "dotenv": "^16.0.0"
  },
  "author": "Your Name",
  "license": "MIT"
}
```

Environment Configuration (.env file)

Create a file called `.env` in your main folder. This file stores your bot's login information. Replace the example values with your actual information:

```
BOT_USERNAME=your_stumblechat_username
BOT_PASSWORD=your_stumblechat_password
PRIMARY_ROOM=room_name_where_bot_appears
```

Important Security Note: Never share your .env file with anyone. It contains your bot's login credentials.

Example of what .env should look like with real information:

```
BOT_USERNAME=MyBotAccount
BOT_PASSWORD=MySecretPassword123
PRIMARY_ROOM=general_chat
```

Step 4: Install Required Software

Open your command prompt or terminal:

- **Windows**: Press Windows key + R, type `cmd`, press Enter
- **Mac**: Press Cmd + Space, type `terminal`, press Enter
- **Linux**: Press Ctrl + Alt + T

Navigate to your project folder by typing:

```bash
cd path/to/your/personality-chatbot
```

For example, if your folder is on your Desktop:

- **Windows**: `cd Desktop/personality-chatbot`
- **Mac**: `cd Desktop/personality-chatbot`

Install the required software by typing this command and pressing Enter:

```bash
npm install
```

This downloads and installs puppeteer (for web browser control) and dotenv (for reading your configuration file). It may take a few minutes to complete.

Step 5: Create Your Bot's Personality (Optional)

If you want to customize your bot's personality, create a file called `personality.js` in the `config/` folder:

```javascript
```

```
// config/personality.js - Your bot's personality configuration

const botPersonality = {
  identity: {
    name: "YourBotName",
    displayName: "Your Bot's Display Name",
    pronouns: "they/them",
    background: "A brief description of who your bot is",
    purpose: "what your bot is designed to do",
    creator: "YourUsername",
    species: "AI",
    age: "newly created",
    origin: "digital realm"
  },

  emotionalProfile: {
    baseEmotions: {
      happiness: 0.7,     // How happy your bot is (0.0 to 1.0)
      trust: 0.6,         // How trusting your bot is
      love: 0.5,          // How loving your bot is
      protectiveness: 0.8, // How protective your bot is
      curiosity: 0.9,     // How curious your bot is
      empathy: 0.8,       // How empathetic your bot is
      playfulness: 0.6,   // How playful your bot is
      anxiety: 0.2,       // How anxious your bot is
      suspicion: 0.1      // How suspicious your bot is
    },
    emotionalVolatility: 0.3 // How quickly emotions change (0.0 to 1.0)
  },

  communicationStyle: {
    formality: 0.3,         // How formal your bot talks (0.0 = casual, 1.0 = very formal)
    emotiveness: 0.7,       // How emotional your bot's responses are
    verbosity: 0.6,         // How much your bot talks (0.0 = brief, 1.0 = very chatty)
    humor: 0.7,             // How much humor your bot uses
    useEmojis: true,        // Whether your bot uses emojis
    useAsterisks: false,    // Whether your bot uses *actions*
    useSlang: true,         // Whether your bot uses slang
    asksQuestions: true,    // Whether your bot asks questions
    sharesPersonalThoughts: true,    // Whether your bot shares thoughts
    remembersConversations: true     // Whether your bot remembers past conversations
  }
};

const responseTemplates = {
  greetings: {
```

```
    general: [
      "Hello {nick}! Great to see you here!",
      "Hey {nick}! How are you doing today?",
      "Hi there {nick}! Welcome!"
    ],
    newUser: [
      "Welcome {nick}! I'm {botName}, nice to meet you!",
      "Hey {nick}! First time here? I'm {botName}!"
    ],
    returning: [
      "Hey {nick}! Good to see you again!",
      "Welcome back {nick}! How have you been?"
    ]
  },

  emotional: {
    supportive: [
      "{nick}, I'm here for you. Want to talk about what's bothering you?",
      "That sounds tough {nick}. I'm listening if you need to talk."
    ],
    celebratory: [
      "🎉 {nick}! That's amazing news! I'm so happy for you!",
      "Woohoo {nick}! That's fantastic!"
    ],
    affectionate: [
      "{nick}, you bring such joy to this place!",
      "You're awesome {nick}!"
    ]
  }
};

const behaviorPatterns = {
  responseProbabilities: {
    randomConversation: 0.05,    // Chance of random responses (5%)
    reactToEmotions: 0.7,        // Chance of responding to emotions (70%)
    greetNewUsers: 0.8,          // Chance of greeting new users (80%)
    askFollowUpQuestions: 0.4,   // Chance of asking follow-up questions (40%)
    offerHelp: 0.6               // Chance of offering help (60%)
  },

  emotionalReactions: {
    detectionKeywords: {
      sadness: ["sad", "depressed", "down", "upset", "crying", "hurt"],
      happiness: ["happy", "excited", "great", "awesome", "amazing", "love"],
      anger: ["angry", "mad", "furious", "annoyed"],
      fear: ["scared", "afraid", "worried", "anxious", "nervous"]
    }
```

```
  }
};

module.exports = {
  botPersonality,
  responseTemplates,
  behaviorPatterns
};
```

If you don't create this file, your bot will use default personality settings.

Step 6: Run Your Bot (Basic Version)

Your bot is now ready to run! In your command prompt/terminal (still in your project folder), type:

```bash
npm start
```

This starts your bot. You should see messages like:

```
🤖 Initializing Personality Engine...
✅ Configuration validated
🔐 Navigating to StumbleChat login...
✅ Login successful!
🤖 PersonalityBot online and ready!
```

If you see error messages instead, check Step 7 below for troubleshooting.

Step 7: Troubleshooting Common Problems

Problem: "BOT_USERNAME not configured" error

Solution: Make sure your .env file exists and has the correct information. Check that there are no extra spaces around the = signs.

Problem: "Login failed" error

Solution:

- Double-check your username and password in the .env file
- Make sure your StumbleChat account exists and can log in through a web browser
- Check that you're not already logged in to StumbleChat in another browser tab

Problem: "Cannot find module" error

Solution: Run `npm install` again to make sure all required software is installed.

Problem: Bot doesn't respond to messages

Solution:

- Check that the room name in your .env file is exactly correct (case-sensitive)
- Make sure you have permission to talk in that room
- Wait a few minutes after the bot starts before testing

Problem: "ENOENT" file not found error

Solution: Make sure you're running the command from the correct folder (the one containing package.json).

Step 8: Adding Advanced Features (Optional)

Your bot works perfectly with just the basic setup. However, you can unlock much more powerful features by activating the modules you placed in the modules folder.

To activate MessageProcessor (Better message handling):

1. Open `src/core/PersonalityEngine.js` in your text editor
2. Find this line: `// const MessageProcessor = require('../modules/MessageProcessor');`
3. Remove the `//` at the beginning to make it: `const MessageProcessor = require('../modules/MessageProcessor');`
4. Find this section:

```javascript
/*
try {
  this.messageProcessor = new MessageProcessor(this, {
    pollingInterval: 2500,
    maxRecentMessages: 100,
    duplicateWindow: 30000
  });
  console.log('✅ MessageProcessor initialized');
} catch (error) {
  console.log('⚠️ MessageProcessor not available, using built-in processing');
}
*/
```

5. Remove the `/*` at the beginning and `*/` at the end
6. Save the file and restart your bot

To activate other modules:

Follow the same process for each module you want to activate:

- `OwnerPrivilegeSystem` - Advanced owner controls
- `EnhancedCommandHandler` - More powerful commands
- `MusicSystem` - Music request capabilities
- `AdvancedSelfAwareness` - AI consciousness simulation

Each module adds new capabilities without breaking existing functionality.

Step 9: Testing Your Bot

Once your bot is running, test it by going to the chat room and typing:

- `.ping` - Bot should respond with a message
- `.help` - Bot should show available commands
- `hello` - Bot should greet you (if personality system is working)
- `.status` - Bot should show its current status

Step 10: Stopping Your Bot

To stop your bot, go back to the command prompt/terminal where it's running and press `Ctrl + C` (Windows/Linux) or `Cmd + C` (Mac).

Understanding What Each File Does

Core Files:

- **PersonalityEngine.js** - The main bot code that handles everything
- **package.json** - Lists what software your bot needs
- **.env** - Stores your bot's login information (keep this private!)

Module Files (Optional Enhancements):

- **MessageProcessor.js** - Makes message detection much more reliable
- **OwnerPrivilegeSystem.js** - Gives bot owners special privileges and security
- **EnhancedCommandHandler.js** - Adds many more commands and smart features
- **MusicSystem.js** - Lets your bot play music and manage playlists
- **AdvancedSelfAwareness.js** - Makes your bot seem more conscious and self-aware

Configuration Files:

- **personality.js** - Defines how your bot behaves and talks (optional)

Bot Commands (Default)

Your bot responds to these commands by default:

- `.ping` - Check if bot is working
- `.help` - Show available commands
- `.mood` - Show bot's current emotional state
- `.personality` - Show bot's personality description
- `.info` - Show relationship info with the user
- `.status` - Show bot system status (owner only)
- `.grant <username>` - Give user special privileges (owner only)
- `.revoke <username>` - Remove user privileges (owner only)

Customizing Your Bot

Changing Personality

Edit the `config/personality.js` file to change how your bot behaves. Higher numbers (closer to 1.0) mean more of that trait, lower numbers (closer to 0.0) mean less.

Adding Custom Responses

In `personality.js`, you can add new response templates to make your bot say different things.

Changing Bot Behavior

Adjust the `responseProbabilities` in `personality.js` to make your bot more or less likely to respond to different situations.

Security and Safety

Important Security Rules:

1. **Never share your .env file** - It contains your login credentials
2. **Use a dedicated bot account** - Don't use your personal StumbleChat account
3. **Monitor your bot** - Check on it regularly to make sure it's behaving properly
4. **Respect chat rules** - Make sure your bot follows the rules of any room it joins

If Something Goes Wrong:

- Stop your bot immediately with Ctrl+C
- Check the error messages in your terminal
- Consult the troubleshooting section above

- Restart your bot after fixing any issues

Getting Help

If you encounter problems not covered in this guide:

1. **Check the error messages** - They often tell you exactly what's wrong
2. **Verify your .env file** - Most problems come from incorrect login information
3. **Make sure Node.js is installed** - Run `node --version` to check
4. **Restart everything** - Sometimes a simple restart fixes mysterious problems

File Organization Summary

When everything is set up correctly, your folder should look like this:

```
personality-chatbot/
├── src/
│   ├── core/
│   │   └── PersonalityEngine.js        # Main bot file
│   └── modules/                        # Optional enhancement modules
│       ├── EnhancedCommandHandler.js
│       ├── MusicSystem.js
│       ├── MessageProcessor.js
│       ├── OwnerPrivilegeSystem.js
│       └── AdvancedSelfAwareness.js
├── config/
│   └── personality.js                  # Your bot's personality (optional)
├── logs/                               # Log files (created automatically)
├── node_modules/                       # Installed software (created by npm install)
├── package.json                        # Project configuration
├── package-lock.json                   # Dependency lock file (created automatically)
└── .env                                # Your bot's credentials (keep private!)
```

Your bot is now ready to run and can be customized to match any personality or behavior you want!